

Agentic Pilot: An Evidence-Driven Local Autonomous AI Agent Framework for Privacy-Preserving Intelligent Task Automation

[TODO: Author name(s)]
[TODO: Affiliation, city, country]
[TODO: email]

Abstract—LLM-based assistants that operate software for a user often send screenshots and page content to hosted models, often keep no memory across sessions, and can declare a task finished when it is not. We present Agentic Pilot, an open-source agent that runs its language and vision models locally through Ollama and drives Chromium through Playwright. The agent is a LangGraph state machine with separate observation, planning, execution, verification and recovery nodes, and it may report a task complete only when a completion predicate evaluated on the live page holds; evaluations are logged next to per-action before/after screenshots. Grounding uses Document Object Model (DOM) elements first and a local vision model as fallback; calls are routed to small models by role, CAPTCHAs halt the task, and high-risk intents need approval. We formalize the gate and define a false completion rate. On 38 constructed browser end states, the predicate withheld completion in 15 of the 23 states that did not meet their goal, all of which a dispatch-only rule accepts; inside the live verification node, where the planner’s own reasoning can stand in for an answer, it withheld 13 to 14. It wrongly rejected 3 of 15 satisfied states, at a median 35.2ms per decision, and each miss points to a missing check. A code audit reports which mechanisms are live. We also design an extension to files, processes and native applications with accessibility-tree grounding, typed postconditions and deterministic permission tiers, and give an end-to-end evaluation protocol for local hardware.

Index Terms—autonomous agents, large language models, local inference, privacy-preserving computing, task automation, execution verification, human-in-the-loop

I. INTRODUCTION

Assistants built on large language models (LLMs) increasingly act on a user’s behalf: they open pages, fill forms and read results. Several of the web and GUI agents reviewed in Section II send each observation, often a full screenshot, to a hosted multimodal model [1]–[4]. For a personal assistant this is a poor fit. The screen of a logged-in user carries names, messages and account data, and leakage of personal information from language models has been measured even after scrubbing [5]. Few of these systems keep memory across sessions, and most operate either in a browser or in one other environment rather than across the applications a person uses (Table I).

A second problem is independent of where the model runs. An agent can believe it has finished when it has not. We call this *premature completion*: the agent emits a terminal “done” while the environment does not satisfy the user’s

goal. A typical case is a search request in which the agent opens the search engine, sees the page load without error, and stops before any query is issued. Every individual browser call succeeded; the task did not. Benchmarks that check final environment state show how wide this gap is [6], [7], and self-assessment by the model is not a reliable remedy, since intrinsic self-correction without external feedback does not improve, and can degrade, reasoning accuracy [8].

This paper describes Agentic Pilot, an open-source agent that runs its language and vision models locally through Ollama and drives Chromium through Playwright. Its central rule is that a task may be reported complete only when a check on the observed environment passes, with the inputs to that check written to disk as evidence. Observation, planning, execution, verification and recovery are separate nodes of an explicit LangGraph state machine, so every transition can be inspected and logged.

We make the following contributions:

- 1) An evidence-gated execution model that separates action dispatch from task completion, formalized as a completion predicate over observations, together with a *false completion rate* (FCR) metric for measuring premature completion (Section IV-C).
- 2) The architecture and implementation of a local agent that combines this gate with role-based routing across local models, grounding in the page’s Document Object Model (DOM) first with a vision-model fallback, CAPTCHA detection that halts rather than bypasses, a task-level human approval gate, and persistent per-step evidence (Sections III–V).
- 3) A controlled, reproducible study of the completion gate on 38 constructed browser end states, run both on the predicate alone and inside the live verification node, which quantifies the premature completions it prevents, the ones it misses and the correct completions it rejects (Section VII).
- 4) A design for extending the framework from the browser to the local system (files, processes and native applications) with accessibility-tree-first grounding, typed desktop verification predicates and deterministic permission tiers; we state precisely which parts of it exist in code today (Section IV-G).
- 5) A code-level audit of which mechanisms are active on

the live execution path, and an end-to-end evaluation protocol with ablations to be run on local hardware (Sections V and VI).

Sections II–IV cover related work, architecture and method; Sections V–VII the implementation, setup and results; Sections VIII–X security, limitations and conclusions.

II. RELATED WORK

a) Reasoning-and-acting agents: ReAct interleaves free-text reasoning with tool actions and observations [9], building on chain-of-thought prompting [10]. Toolformer shows that a model can learn when to call an external API [11]. Reflexion stores verbal self-critiques in an episodic buffer and retries with them [12]; RCI applies recursive self-criticism to computer tasks [13]; Voyager accumulates a skill library and uses a model-based self-check before committing a skill [14]. In all of these, the decision that a task is finished is made by the model, sometimes with a critic that is also a model. Huang et al. find that such intrinsic self-correction does not reliably improve reasoning without external feedback [8]. Agentic Pilot moves the completion decision out of the model and into a check on the observed page.

b) Web agents and benchmarks: Early web agents were trained with reinforcement or imitation learning on MiniWoB-style pages [15]. Mind2Web pairs a small ranking model that filters candidate DOM elements with an LLM that chooses the action [16], and WebArena evaluates agents on self-hosted sites with functional correctness checks [6]. WebVoyager drives live sites from screenshots and uses a multimodal model to judge saved trajectories after the episode [1]. The correctness checks in these works belong to the benchmark and run after the episode ends. Our gate runs inside the agent at the moment it tries to stop.

c) Desktop, OS and mobile GUI agents: OSWorld provides execution-based checkers for tasks in real operating systems and reports a large gap between agents and people [7]. UFO controls Windows applications through UI Automation and asks the user to confirm sensitive actions [3]. AppAgent learns reusable documentation of smartphone apps [4], and GPTDroid and QTypist apply LLMs to mobile GUI testing and text-input generation [17], [18]. These systems either rely on hosted models or target testing rather than verified completion of user tasks. Our desktop extension (Section IV-G) borrows UFO’s accessibility-first control and OSWorld’s state-based checking, but places the checks inside the agent loop.

d) Visual grounding for GUI interaction: SeeAct finds that grounding a chosen action onto the right element is the main bottleneck for multimodal web agents, and that combining HTML with the image outperforms Set-of-Mark overlays [2], [19]. CogAgent and SeeClick train vision-language models that locate elements from screenshots alone [20], [21], and OmniParser converts screenshots into structured element lists [22]. These results motivate our ordering: structured DOM grounding first, pixel-level grounding only when the DOM route fails.

TABLE I

CAPABILITIES REPORTED BY REPRESENTATIVE SYSTEMS. LOC: RUNS ON OPEN OR LOCALLY HOSTABLE MODELS; WEB/DESK: TARGET ENVIRONMENT; GATE: COMPLETION GATED AT RUN TIME BY AN IN-AGENT ENVIRONMENT CHECK; EVID: PERSISTS STATE ARTIFACTS; MEM: CROSS-STEP OR CROSS-EPISEODE MEMORY; INJ: PROMPT-INJECTION DEFENSE; APPR: HUMAN APPROVAL FOR RISKY ACTIONS. **Y** REPORTED, **P** PARTIAL OR WEAKER FORM, **N** NOT SUPPORTED, – NOT REPORTED. THE LAST ROW REFLECTS THE AUDITED CODE (SECTION V).

System	Loc	Web	Desk	Gate	Evid	Mem	Inj	Appr
ReAct [9]	N	P	N	N	N	N	N	N
Reflexion [12]	–	P	N	P	N	Y	N	N
WebArena [6]	–	Y	N	N	–	N	N	N
Mind2Web [16]	Y	Y	N	N	P	N	N	N
WebVoyager [1]	N	Y	N	N	Y	N	N	N
SeeAct [2]	P	Y	N	N	–	N	N	P
OSWorld [7]	Y	Y	Y	N	–	N	N	N
UFO [3]	N	P	Y	–	–	P	N	Y
AppAgent [4]	N	N	N	–	–	Y	N	N
CogAgent [20]	Y	Y	–	N	N	N	N	N
MemGPT [26]	–	N	N	N	N	Y	N	N
AgentDojo [30]	Y	N	N	N	–	N	Y	–
Agentic Pilot	Y	Y	P ^a	P ^b	Y	P ^c	P ^d	P ^e

^aDesigned (Section IV-G); a prototype executor exists but is not wired into the agent. ^bRule-based predicate Φ , weakened on the live path (Section VII). Benchmark-side checkers (WebArena, OSWorld, AgentDojo) and post-hoc judges (WebVoyager) are not in-agent gates. ^cOutcomes are written; retrieval is not active on the live path. ^dPattern denylist on element text. ^eTask-level, triggered by the intent’s action type or model-assigned risk level. AppAgent targets Android, which we do not count as desktop.

e) Agent memory and retrieval: Retrieval-augmented generation couples a model with a non-parametric store [23], usually indexed with sentence embeddings [24]. Generative Agents score memories by recency, importance and relevance [25], and MemGPT pages context between tiers in the manner of an operating system [26]. None of these ties a stored memory to verified evidence of the outcome it describes, which is what an execution agent needs if it is to learn from its own runs.

f) Security of LLM agents: Indirect prompt injection places instructions in content the agent reads, such as a web page [27]. Liu et al. formalize such attacks and benchmark defenses [28]; InjecAgent and AgentDojo measure how often tool-using agents follow injected instructions [29], [30]. A browser agent reads untrusted text at every step, so it needs both input filtering and a policy that keeps high-impact actions under human control.

g) Local inference and surveys: Open-weight families such as Qwen2.5 and Qwen2-VL provide text and vision models at sizes that run on consumer hardware [31], [32], and weight quantization further reduces their memory footprint [33]. Surveys of LLM agents list trustworthiness among the open problems [34], [35]. How often models of this size finish GUI tasks whose outcome is independently checked has received little measurement, and our evaluation protocol targets that question.

Table I summarizes the comparison. Cells record only what each paper reports.

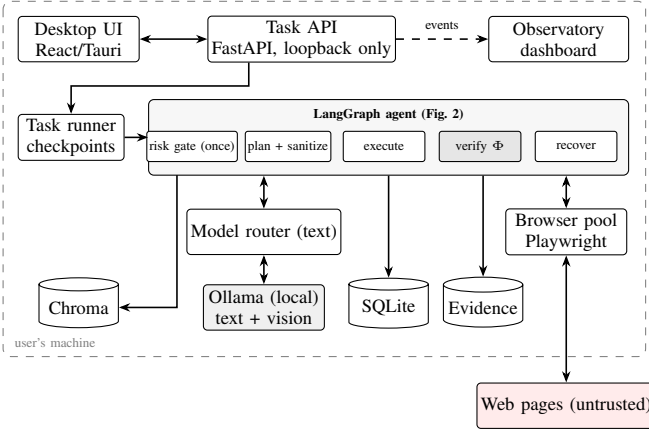


Fig. 1. Components of Agentic Pilot. Everything inside the dashed boundary runs on the user’s machine, including model inference. Web pages are the main untrusted input, and browsing is the main outbound traffic. The Observatory also reads the task database and evidence files. Memory records are written to Chroma but not read on the live path.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the components and the trust boundary. A desktop client submits a natural-language task to a FastAPI service bound to the loopback interface. The task runner queues it, executes the agent graph, checkpoints the state after every node, and cancels tasks that exceed a wall-clock limit (5 min by default). Checkpoints are used when a task is re-scheduled after approval or a CAPTCHA; a task interrupted by a crash is not resumed.

The agent is a compiled LangGraph state machine with eleven nodes (Fig. 2). A typed state record carries the parsed intent, the plan and step index, the page manifest, the planned action, the action history, retry and model-call counters, routing decisions and the status; nodes return partial updates and the routers read only the status and a few flags. The single terminal node, `complete`, persists the outcome, writes memory records and decides whether the browser stays open for the user. Text-model calls go through a router (Section IV-E), and all model calls through a gateway that requests JSON output from a local Ollama server, validates it against Pydantic schemas and re-prompts on malformed output. A pool of Playwright contexts over one Chromium instance, which runs with a visible window by default, gives each task its own page. Task events go to SQLite and to a broadcaster; the Observatory, a separate process, subscribes to that stream and reads the database and evidence files to display plans, routing decisions, verification outcomes and screenshots. It only issues read queries and has no command channel to the agent.

Two inputs cross into the agent: the user’s task text, which is trusted, and page content, which is not. Page content reaches the models as element text, page title and URL, and, on the vision path, screenshots. Apart from one-time downloads of model weights (the Ollama models and, on first use, the default embedding model of the vector store), and, for older versions of the vector-store library, anonymous usage telemetry that

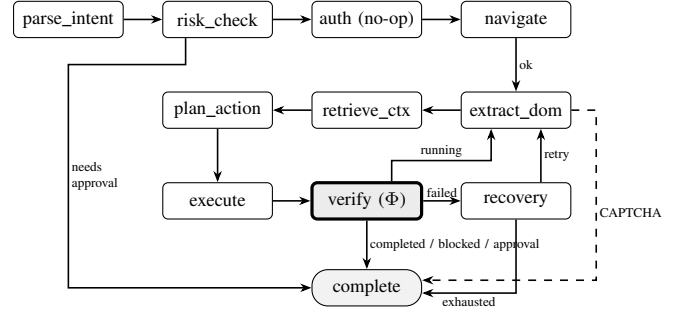


Fig. 2. Execution graph compiled in `graph.py`. Only the shaded `verify` node can end a task as `COMPLETED`, and only when Φ holds. A CAPTCHA seen during observation ends it as `BLOCKED`. Not drawn: failures in `parse_intent` and `navigate` also route to `recovery`.

the code does not disable, outbound traffic in the default configuration consists of the browser’s requests to the sites a task visits; model traffic stays on the local host unless the user points the Ollama address elsewhere.

IV. METHODOLOGY

A. Task Decomposition and Planning

The `parse_intent` node asks a lightweight model for a structured intent: action, target site, target and content strings, risk level and confidence. If the request contains a connective (“and”, “then”, “after”, a comma or semicolon) or more than eight words, a planning model decomposes it into steps $P = (g_1, \dots, g_n)$, each with a description, an action type and an expected outcome; if a permissive pattern (“type ...”, “enter ...”) finds literal text to enter, the plan is replaced by a single text-entry step, which can misfire on ordinary requests containing these words; anything else becomes a one-step plan.

Execution then chooses one action per iteration. `plan_action` gives the model the goal, the plan with the current step marked, the page title, URL and state, and the element manifest, and asks for one of `navigate`, `click`, `type_text`, `press_key`, `scroll`, `extract`, `complete` or `need_help`. Deterministic rules then adjust the choice: exact-text requests are grounded to the best-scoring visible text field, a repeated navigation to the current site is turned into a failure that enters recovery, and plan steps that name clicking a result, extracting or reporting are mapped to those actions. Some rules are site-specific; on Google with a search goal the agent types the query and presses Enter rather than loading a results URL directly (see Section VIII).

B. Grounding: DOM First, Vision on Demand

Each observation starts with a DOM pass over buttons, inputs, text areas, selects, links, elements with button or link roles, and editable regions. For each element the extractor records tag, role, accessible label, up to 120 characters of text, placeholder, type, visibility, enabled state, bounding box, and three selectors: an identifier attribute that the extractor writes into the live DOM, a CSS selector and an XPath. The executor

uses the first that is set, which in practice is always the injected identifier. Only interactable elements are kept; they are ranked with submit-like controls first, deduplicated, limited to five links, stripped of cookie and advertising labels, and capped at 50.

The vision model is used only when the planner returns `need_help`, by choice or because its call failed. The agent sends a screenshot and receives an action with a target point given as fractions of the 1280×800 viewport, where it clicks and, if needed, types. Such actions are not checked individually; a wrong click is caught, if at all, by the completion predicate. The vision model receives the intent’s action verb and target, not the full request, and identical inputs are served from a cache.

C. Evidence-Gated Verification Model

We model an episode as a sequence of browser states s_t with observations $o_t = \Omega(s_t)$ (URL, title, element manifest, screenshot). The planner maps the goal g , o_t and retrieved context to an action a_t ; the executor applies it and returns a dispatch flag $\alpha_t \in \{0, 1\}$, which is 1 when the browser call raised no exception and, for text entry, the value read back matched. Some fallback paths (a mouse-and-keyboard retry of text entry, vision-coordinate clicks) return $\alpha_t = 1$ without such a check.

a) *Blocking predicate*: $\beta(s) = 1$ when the page is a bot check: the URL contains a challenge path, the title contains *captcha*, *robot* or *sorry...*, the body contains one of 12 challenge phrases, or a challenge frame is present. This check runs before Φ and ends the episode as **BLOCKED**.

b) *Step and task predicates*: The decomposer produces a plan $P = (g_1, \dots, g_n)$. The per-step predicate is currently weak, $\phi_i := \alpha_i$: a step is marked complete when its action dispatches. Completion is decided by $\Phi(g, s_{t+1}, \hat{y}, P)$, evaluated on the live page after the action. With $\hat{y}$ the answer,

$$\Phi = \neg E(s) \wedge \neg H(g, s, P) \wedge T(g, s) \wedge X(g, \hat{y}) \wedge R(P, \hat{y}), \quad (1)$$

where E holds for a browser-error URL or a title containing one of four HTTP error phrases (404, 500, 502, 503); H holds when the request contains a search or extraction keyword, or the plan has more than one step, and the URL is the bare home page of one of four search engines; T requires that the literal text extracted from g by a pattern (“enter exactly ...”, “type ...”) appears, as a substring, in an input, text area or editable element; X , instantiated by extraction keywords, requires a non-empty structured extraction or an answer longer than 40 characters that does not begin with a navigation verb; and R requires that no plan step is pending unless an answer exists. H , T , X and R are instantiated only when their triggers are present; for a request that triggers none of them, Φ reduces to $\neg E$.

c) *When Φ is evaluated*: Φ is evaluated when the planner emits `complete`, when the plan index has reached n , or when a requested text entry has dispatched. For a one-step plan, the default for short requests, this is after every successful

action, so Φ also acts as the termination rule. The answer is $\hat{y} = y_{\text{ext}}$ if an extraction has produced one, otherwise the planner’s `reasoning` string for the last action, which makes X and R easy to satisfy (Section VII).

d) *Completion rule*: The agent reports success only if $\Phi = 1$; each evaluation is written to the evidence store with its expected and observed values. A passing evaluation can still yield **WAITING APPROVAL**, and the terminal node downgrades a success to **FAILED** if the intent named a site but no navigation was verified. A rejected evaluation does not end the episode: control returns to observation and planning until a counter of intent, decomposition and planning calls reaches $K = 15$.

e) *Consequence*: Dispatch success does not imply reported success, $\alpha_t = 1 \not\Rightarrow \Phi = 1$; a successful `navigate` to a search engine’s home page for a search request is an example ($H = 1$). The converse guarantee, that $\Phi = 1$ implies the goal is met, does not hold, and Section VII measures how often it fails.

f) *False completion rate*: Let $\mathcal{T}_c$ be the episodes reported complete and Φ^* an independent, task-specific oracle written by the experimenter. Then

$$\text{FCR} = \frac{|\{\tau \in \mathcal{T}_c : \Phi^*(\tau) = 0\}|}{|\mathcal{T}_c|}. \quad (2)$$

FCR is a precision-type measure; an agent that never completes has no false completions. It must therefore be reported together with task success and the false-rejection rate, the fraction of goal-satisfying episodes not reported complete.

D. Failure Classification and Recovery

A failed action is first retried through a fresh observation by the `verify` node, up to three times. Later failures go to the recovery node, which matches the error text against patterns for seven types (`captcha`, `transient`, `element_not_found`, `verification_failed`, `navigation_failed`, `vision_needed`, `llm_failure`) or `unknown`. A **CAPTCHA** ends the task as **BLOCKED**. Other failures are assigned a strategy from the ladder `retry` $\rightarrow$ `alternative_selector` $\rightarrow$ `vision_fallback` $\rightarrow$ `replan`, one level per two attempts, with type-specific shortcuts; after six attempts the task fails. Each decision is stored as a recovery record. Because the retry counter is shared with the `verify` node, the engine usually starts at the second level, and `replan` is reached only through type-specific shortcuts. In the current code the chosen strategy is also not executed: its state field is not declared in the schema and is dropped, so every non-terminal recovery re-observes and re-plans.

E. Role-Based Local Model Routing

A deterministic analyzer maps each call’s type, text and flags to required capabilities and a role: recovery calls to a recovery model; image inputs, or text containing screen-related keywords as substrings, to a vision model; intent parsing and

simple navigation to a lightweight model; decomposition and multi-clause requests to a planner; the rest to a general model. Each role has a configured, ordered candidate list; the defaults are small Qwen2.5, Qwen3.5 and DeepSeek-R1 models of 1.5B–7B parameters for text roles and qwen3-vl:2b or moondream for vision. The first candidate the local Ollama server reports as installed is used; the installed list is refreshed at the start of each task. To limit model reloads, the router keeps the active model, or its own last choice, whenever it has the required capabilities, except for recovery calls; with no installed candidate it falls back to a fallback and then a base model. A single-model mode serves as an ablation. A switch cool-down is configurable but not read, the recovery role is routed although recovery makes no model call, and the vision fallback chooses its model from its own list without the router.

F. Episodic Memory

On termination, `complete` writes the outcome to SQLite and to a Chroma collection: for a success, the goal, final URL and number of model calls; for a failure, the error and URL. The retrieval side exists: a context router queries memory (and, for longer or procedural tasks, a knowledge collection), and a builder deduplicates the results and fits them into a 3,500-token budget in the planning prompt, with knowledge (but not memory) wrapped in an untrusted-content banner. On the live path, however, the runner initializes the retrieved lists as empty rather than unset, which the router treats as already retrieved; memory is written but not read, and knowledge retrieval does not run either. Since the stored records hold no selectors or action sequences, repairing the read path is necessary but not sufficient for learning from past runs.

G. Extension to Local System Access

The loop carries over to the desktop if the accessibility tree replaces the DOM, a window screenshot the page screenshot, and typed operating-system postconditions the URL and DOM checks. Today the repository has only a desktop executor wrapping PyAutoGUI (click, type, hotkey, screenshot) and psutil (process list) that no node, action or API route calls and whose only test checks that its methods exist, plus an unconfined file helper that the agent does not use. The rest of this subsection is a design (PROPOSED).

a) Actions, grounding and verification: Actions cover files (read, hash, create, overwrite with backup, move, trash, delete), processes (launch an allow-listed program, terminate), windows (open, focus, close) and controls (invoke, set value, type). Grounding reads the foreground window’s accessibility tree (Windows UI Automation, macOS Accessibility, AT-SPI) into a manifest shaped like the DOM manifest, with vision on a window screenshot as fallback. Every mutating action carries exact postconditions checked after execution and stored with before/after values (Table II); processes are identified by PID and creation time to guard against PID reuse, and a coordinate action counts as done only when its postcondition holds.

TABLE II
PROPOSED DESKTOP POSTCONDITIONS AND DEFAULT PERMISSION TIERS (DESIGN, NOT IMPLEMENTED). h_0 : HASH CAPTURED BEFORE THE ACTION; (pid, t_c) : PROCESS ID AND CREATION TIME.

Action	Postcondition checked after execution	Tier
create file	file exists $\wedge$ sha256 = sha256(content)	T1
overwrite	backup of pre-image stored $\wedge$ hash equals new content	T1
move	source absent $\wedge$ sha256(dst) = h_0 (src)	T1
move to trash	path absent $\wedge$ trash entry with that origin	T1
permanent delete	path absent	T2
launch app	pid alive with creation time t_c $\wedge$ window title matches	T1/T3
terminate	(pid, t_c) gone within timeout	T1/T2
set value	accessibility value of element equals input	T1/T3

b) Permission tiers and enforcement: Tiers are computed from the concrete action, never from the model’s risk label. T0 (read-only) runs freely; T1 (reversible, with a recorded inverse) runs automatically only if the user enabled automatic approval; T2 (destructive, e.g., permanent deletion or terminating a process the agent did not start) and T3 (network-capable programs, password fields) always need approval, and secrets never pass through the model. Paths outside allow-listed roots, inline-code interpreters and privilege elevation are refused. A new `action_gate` node between planning and execution applies the tier, a path jail that resolves symbolic links before testing containment, and a dry-run preview. Approvals are bound to a hash of the exact action, and each action is appended to a hash-chained audit log that records digests, never contents.

V. IMPLEMENTATION

The backend is Python (10,755 lines, 1,682 of them in the graph nodes) on FastAPI, LangGraph, Playwright and Pydantic, with SQLite for tasks and events and a persistent local Chroma store for vectors. The client is React in a Tauri shell; the Observatory is a separate FastAPI service with a React dashboard. The code is public at <https://github.com/sudeshsudhii/Agentic-pilot-local>. **[TODO: AUTHOR: confirm license; archive the evaluated version (tagged release with DOI).]**

a) Design decisions: The runner records each node’s name and status in a trace and checkpoints the state after every node, so a task re-scheduled after an approval or a solved CAPTCHA re-enters the graph with its saved plan and skips intent parsing. The completion decision lives in one node; the terminal node adds a guard that downgrades a completion to FAILED if the intent named a site but navigation was never verified. Planner output always passes through deterministic rules, which corrects small models on known patterns but places part of the behavior outside the model.

TABLE III

STATUS OF EACH MECHANISM AT COMMIT 2361d50, TRACED THROUGH THE CODE. I+T: IMPLEMENTED AND UNIT-TESTED; I: IMPLEMENTED, UNTESTED; P: PROPOSED OR PARTIAL. LIVE: AFFECTS THE DEFAULT EXECUTION PATH.

Mechanism	Status	Live	Note
Graph, checkpoints, re-scheduling	I+T	yes	graph run only by e2e tests
Completion predicate Φ	I+T	yes	weaker live (Sec. VII)
Per-step predicates, click check	P	–	steps advance on dispatch
CAPTCHA halt, resume, fallback	I+T	yes	no solver
DOM grounding, vision fallback	I+T	yes	vision on <code>need_help</code> only
Failure classes, retry bounds	I+T	yes	7 types; 3+3 attempts
Strategy-specific recovery	P	no	strategy label dropped
Role routing, stickiness	I+T	yes	substring keywords; no cool-down
Memory write / RAG retrieval	I / I+T	yes / no	retrieval bypassed
Sanitizer on element text	I+T	yes	title, URL unfiltered
Task-level approval gate	I	yes	once per task
Model-call audit log	I+T	yes	browsing not logged
Evidence store	I+T	yes	some files overwritten
Ablation flags	I	partial	verification flag unread
Desktop executor / extension	I / P	no	not wired (Sec. IV-G)

b) Evidence: Per task, the evidence directory holds before/after screenshots of each executed action, screenshots after navigation and on verified completion, the latest DOM manifest, accumulated verification and recovery records (rewritten on each update, and reset if the file fails to parse), a per-action execution record (action, URLs, screenshots, result, model, role, duration) and the node trace. A retry at the same model-call count overwrites the earlier screenshot pair, a rejected Φ evaluation keeps no page snapshot of its own, and files carry no integrity protection. Typed text is stored in clear in task events and execution records.

c) Status audit: Agent documentation tends to run ahead of the code, so we traced each mechanism to the path that executes it (Table III). Most are live. Four are implemented but inert in a normal run: recovery strategies other than re-planning, memory retrieval, the routing cool-down and the verification ablation switch. The repository’s evaluation command calls scripts that are not in the repository, its ablation runner imports a function that does not exist, and its certification report counts a task as passed when it ends as completed and any screenshot and trace file exist. We use none of these repository-reported figures as results. The extraction handler, finally, was written for one case study: it formats every extraction as facts about a particular university campus and, when the page yields too few matching lines, substitutes fixed text. It can therefore produce a fabricated answer that satisfies X . It must be removed before any end-to-end evaluation.

VI. EXPERIMENTAL SETUP

A. Component Studies

The gate and router studies need neither a language model nor network access and were run for this paper on a Linux container with a 4-core Intel Xeon at 2.8 GHz, 15 GiB RAM and no GPU, using Python 3.11.15, Playwright 1.56.0 with headless Chromium 141.0.7390.37 and LangGraph 1.2.12, on agent code at commit 2361d50. The study scripts are under `artifacts/experiments/` on the paper’s branch of the repository.

a) Gate study: We built 38 terminal browser states in six categories (Table IV), including two satisfied states that resemble bot checks and one request whose wording triggers no conjunct. Each has a goal, a URL, an HTML document served for it by Playwright request interception (no network traffic), the answer and plan state where relevant, and an oracle label Φ^* with a rationale. Every case ends with a dispatched action, so a dispatch-only rule accepts all of them. We judge each state with the pre-check and `verify_task_completion` called directly (component), and with the real `verify` node, its database, browser-pool and evidence I/O stubbed, on the state the graph would hold after the final action, with an empty or a generic one-sentence planner `reasoning` (live). Latency is the median of 20 repetitions of the component decision after a warm-up. We wrote the cases and labels ourselves, including hard ones, so the rates describe coverage, not expected performance.

b) Routing study: We marked models as installed in the registry and called the router with each call site’s arguments, in isolation and in the agent’s call order with the previous selection as the active model.

B. End-to-End Protocol

a) Hardware and models: Record CPU, GPU, RAM, operating system, Ollama version and model digests. Configurations use the default models (`qwen2.5:1.5b`, `moondream`) or all configured models. The gateway sends no sampling options, so we set temperature 0 and a fixed seed (a small change) and run three trials per task. **[TODO: MEASURE: hardware, OS, Ollama version, model digests.]**

b) Task suite: The repository’s eight benchmark tasks include four without a checkable criterion. We propose 30 tasks, five in each of six categories: navigation, search, form text entry, extraction of a known fact, two- to three-step tasks, and blocking or error conditions (a public CAPTCHA demonstration page, HTTP error endpoints). Each has an oracle Φ^* written before any run and applied to the retained page or the saved final screenshot by a person blind to the agent’s reported status. **[TODO: AUTHOR: finalize and publish the 30 tasks and oracles.]**

c) Configurations: (i) Full system. (ii) No gate: Φ replaced by “accept” (requires the `verify` node to honor the unused `enable_verification` flag). (iii) No recovery (supported). (iv) Vision-only grounding: the element manifest withheld from the planner, while Φ and the rules keep DOM

TABLE IV

DECISIONS ON 38 CONSTRUCTED END STATES. n^+/n^- : STATES THAT DO / DO NOT SATISFY THE GOAL UNDER THE ORACLE. FC: FALSE COMPLETIONS (REPORTED COMPLETE, GOAL NOT SATISFIED); FR: FALSE REJECTIONS (GOAL SATISFIED, NOT REPORTED COMPLETE). A DISPATCH-ONLY RULE HAS FC = n^- AND FR = 0. COMP.: CAPTCHA PRE-CHECK AND Φ CALLED WITH THE CASE’S ANSWER. LIVE: THE REAL VERIFY NODE WITH AN EMPTY (E) OR A GENERIC ONE-SENTENCE (V) PLANNER REASONING STRING. SOURCE: VERIFICATION_GATE_SUMMARY.CSV.

Category	n^+	n^-	Comp.		Live E		Live V	
			FC	FR	FC	FR	FC	FR
Search	3	4	1	0	1	0	1	0
Navigation	2	6	3	0	3	0	3	0
Bot check	2	4	1	2	1	2	1	2
Text entry	3	3	1	0	1	0	1	0
Extraction	3	4	1	1	1	1	2	1
Multi-step	2	2	1	0	2	0	2	0
All	15	23	8	3	9	3	10	3
FCR		0.605*		0.400		0.429		0.455

*Dispatch-only rule.

access (a small change). (v) Single model (supported). A memory ablation is meaningful only once retrieval is repaired (Section IV-F).

d) *Metrics*: Success rate under Φ^* ; FCR (Eq. 2) and false-rejection rate; model calls and executed actions; wall-clock time; recovery success (recovered tasks that end satisfied); vision-invocation rate; and evidence completeness (executed actions with both screenshots and an execution record, and completed tasks with a completion screenshot). Rates carry Wilson 95% intervals over task-trial pairs.

VII. RESULTS AND DISCUSSION

A. Completion Gate on Constructed End States

Table IV reports the gate study. Called directly, the pre-check and Φ withhold completion in 15 of the 23 unsatisfied states that a dispatch-only rule accepts (search home pages, HTTP error titles, three bot checks, empty or wrong text fields, empty, navigational or ungrounded answers, a pending plan step) and reject 3 of the 15 satisfied ones. Inside the live verify node the gate is weaker. The node marks the current step complete as soon as its action dispatches, so a premature complete at the last step passes the plan check (P1), and it uses the planner’s reasoning as the answer when none exists, so with verbose reasoning the empty-answer case passes too (X2). The live gate withholds 14 or 13 of the 23. FCR values depend on our case mix; the case-level outcomes carry the information.

Each remaining false completion has a specific cause. A results page for another query passes because the search check only asks whether the URL is a results page (S7). A soft “page not found” and a login redirect match none of the four error titles (N6, N7). An interstitial with none of the 12 challenge phrases passes (C4). A superset of the requested text passes because matching is by substring (T4). Any answer over 40 characters not starting with one of eight verbs counts as an extraction (X4). A click on the wrong element passes

because step status comes from dispatch (P3). A request whose wording triggers no conjunct reduces Φ to the error check (N8). The false rejections are a correct 13-character answer (X6) and two pages wrongly blocked as bot checks, an article titled “Robot” (C5) and a login page with a hidden invisible-reCAPTCHA frame (C6). None of the missing checks needs a model call: query comparison, exact matching, answer-to-goal overlap, step predicates from the planner’s expected outcome, visibility-aware challenge detection, and an answer field the planner’s reasoning cannot fill. The fixtures can serve as their regression suite.

The gate is cheap next to inference: the 33 decisions that evaluate Φ had a median of 35.2 ms (maximum 52.5 ms; each the median of 20 repetitions), and the five that stop at the pre-check took at most 8.2 ms, mostly spent reading page text through the browser protocol. The fixture pages are small, so real pages will take longer.

B. Routing Behavior

We evaluated the router at each call site without inference (`routing_resolution.csv`). With the default installation (`qwen2.5:1.5b`, `moondream`), intent parsing, decomposition, planning and recovery all resolve to `qwen2.5:1.5b`, three via the fallback chain, but keywords match as substrings, so “find a guitar guide” (“gui”) or “search Google Images” is routed to `moondream`. With all configured models installed, isolated calls get their role’s first candidate. In the agent’s call order, however, stickiness keeps `deepseek-r1:1.5b`, chosen for intent parsing and profiled as able to plan, for decomposition and planning; recovery switches to `qwen2.5:7b` although it makes no model call, and that model is then kept for planning. The router also reuses its last choice across tasks, and the vision fallback bypasses it. In our probes the planner model was never selected for planning.

C. Test Suite

Without Ollama and with a 240s per-test timeout, the repository’s suite collects 108 tests in 24 files: 104 pass, 4 fail and one module fails to import; the 20 Observatory tests pass, and backend line coverage is 62.0%. Passing tests use no language model: node tests stub the executor and gateway, and the end-to-end tests run the compiled graph only through failure paths and accept a failed task. Two failures need Ollama and one a directory absent from the repository; the fourth, `test_action_failure_never_marks_completed`, which targets the central property, times out headless. The suite supports claims about component behavior, not task success.

D. End-to-End Evaluation

End-to-end results need a local Ollama installation and live websites, which the drafting environment lacked. **[TODO: MEASURE: run the protocol of Section VI and report, per configuration, success rate, FCR and false-rejection rate under the oracle, model calls, wall-clock**

time, recovery success, vision rate and evidence completeness, with 95% intervals.] [TODO: MEASURE: add a case study with before/after screenshots from `~/.pilot/logs/evidence/<task_id>/` of a task in which the gate rejected a premature completion and the agent then finished.]

VIII. SECURITY, PRIVACY AND ETHICAL CONSIDERATIONS

a) *Threat model*: We consider (i) instructions embedded in page content that try to redirect the agent [27]; (ii) high-impact actions such as posting, sending or purchasing without consent; (iii) damage to the local system once file and process actions exist; and (iv) disclosure of user data to unintended parties. The user and the host are trusted; websites are not. A compromised host or model is out of scope.

b) *Current mitigations*: A sanitizer replaces 17 patterns of instruction-override, delimiter, exfiltration and role-manipulation phrasing in element text, labels and placeholders before they reach the planner. Before navigation, a risk gate pauses tasks whose parsed action is posting, e-mailing, purchasing, deleting or transferring, or whose risk level is high or critical, and asks the user. The intent and risk level are derived from the user’s request alone, before any page is read, so page content cannot lower them. The service listens on loopback, every model call is logged with destination and payload sizes, and credential-like values are scrubbed from broadcast events.

c) *Gaps*: The sanitizer is a fixed denylist; systematic evaluations find existing prompt-injection defenses insufficient [28], and paraphrased or encoded instructions do not match its patterns. Page title, URL, extracted text, memory records and screenshots are not filtered. Both the action label and the risk level that trigger approval are free text chosen by a small model, and the intent prompt does not offer all the labels the gate checks for. Approval is decided once per task: no individual action is re-assessed after page content has been read, so an injected instruction that steers the planner to submit a form is not gated. The API has no authentication; the loopback binding and a localhost-only CORS policy limit, but do not control, who can call it. The audit log covers model calls but not browsing, and its locality flag is a substring test that is logged, not enforced. The privacy property we claim is therefore narrow: *by default, model inference runs on the user’s machine and no prompt or screenshot is sent to a model provider*. Data the user asks the agent to enter on a website still leaves the machine, and typed text is stored in clear locally. The desktop design addresses several of these gaps with deterministic tiers, per-action and action-bound approvals, and a hash-chained audit log.

d) *CAPTCHAs and automation etiquette*: On a CAPTCHA or bot check the agent stops, marks the task BLOCKED and keeps the browser open; the user can solve the challenge and resume (refused while it is still present) or restart the search on another engine, which loads its result URL directly. There is no solver. The browser does, however, disable automation-detection features, send a fixed desktop

user-agent string, and hide `navigator.webdriver`; on Google it types queries instead of loading result URLs; and the DOM extractor writes identifier attributes into every page, which sites can observe. These measures make the agent look like a person using a browser. We consider them acceptable only for a personal assistant acting at human pace on its user’s own requests, never for bulk access, and we recommend that deployments disable them wherever a site’s terms prohibit automated access. Chromium also runs with its sandbox disabled; re-enabling it is a simple hardening step.

IX. LIMITATIONS AND FUTURE WORK

Only the gate and the router are measured here, the gate on constructed states; end-to-end rates and host-dependent latencies remain to be measured (Section VI). Φ is keyed to request wording and a few search engines, reduces to an error check when no keyword fires, and on the live path can be satisfied by the planner’s own reasoning. We plan typed step predicates derived from the planner’s expected outcomes, an optional local vision-model check of the final screenshot, and calibration on labelled trajectories. Planning with 1.5B–7B models is weak, and deterministic overrides compensate on common patterns; several are site- or case-specific (the Google search rule, a click fallback that matches one site’s name, substring rules that map plan steps to actions, and the extraction handler of Section V). [TODO: AUTHOR: remove the case-specific extraction and click code (nodes .py lines 1052 and 1100–1142), list remaining site-specific rules in an appendix, and state whether any earlier repository figure depended on them.] Canvas- and WebGL-rendered pages depend entirely on the vision path. Strategy-specific recovery, memory retrieval, the routing cool-down and the verification switch need small fixes to take effect (Table III). For the local-system extension we will first implement the path and process policy, the predicate library, an AT-SPI provider (testable headlessly on Linux) and the `action_gate` node; longer term we plan to separate planning, grounding and verification into cooperating agents, with the verifier given no write access.

X. CONCLUSION

Agentic Pilot runs a browser agent against local models and reports success only when a check on the observed page passes. Separating observation, planning, execution, verification and recovery into graph nodes made it possible to state that rule formally and test it in isolation: on constructed end states it withholds most premature completions a dispatch-only agent would report, but inside the live node the planner’s reasoning and dispatch-based step status weaken it, and each miss points to a missing check. The same structure extends to files, processes and applications once accessibility trees, typed postconditions and deterministic permission tiers replace the DOM, page checks and model-assigned risk. What remains is to close those gaps, remove case-specific code, activate the designed recovery and memory paths, and measure the full system on real tasks.

ACKNOWLEDGMENT

[TODO: AUTHOR: review and adapt.] An AI system (Anthropic’s Claude) assisted with analyzing the source code, running the reported component experiments, searching and checking references, and drafting the text of this paper. The authors reviewed, verified and revised all content and take full responsibility for it.

REFERENCES

- [1] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “WebVoyager: Building an end-to-end web agent with large multimodal models,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 6864–6890.
- [2] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, “GPT-4V(ision) is a generalist web agent, if grounded,” in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 61 349–61 385.
- [3] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, “UFO: A UI-focused agent for Windows OS interaction,” in *Proc. 2025 Conf. Nations of the Americas Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT) (Volume 1: Long Papers)*, 2025, pp. 597–622.
- [4] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, “AppAgent: Multimodal agents as smartphone users,” in *Proc. 2025 CHI Conf. Human Factors Comput. Syst. (CHI)*, 2025, pp. 70:1–70:20.
- [5] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Analyzing leakage of personally identifiable information in language models,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 346–363.
- [6] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [7] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.
- [8] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.
- [11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [12] N. Shinn, F. Cassano, G. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [13] G. Kim, P. Baldi, and S. McAleer, “Language models can solve computer tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [14] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Trans. Mach. Learn. Res. (TMLR)*, 2024.
- [15] T. Shi, A. Karpathy, L. Fan, J. Hernandez, and P. Liang, “World of bits: An open-domain platform for web-based agents,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 70, 2017, pp. 3135–3144.
- [16] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a generalist agent for the web,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [17] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 100:1–100:13.
- [18] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, “Fill in the blank: Context-aware automated text input generation for mobile GUI testing,” in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1355–1367.
- [19] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, “Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V,” arXiv:2310.11441, 2023.
- [20] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, J. Tang *et al.*, “CogAgent: A visual language model for GUI agents,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 14 281–14 290.
- [21] K. Cheng, Q. Sun, Y. Chu, F. Xu, L. YanTao, J. Zhang, and Z. Wu, “SeeClick: Harnessing GUI grounding for advanced visual GUI agents,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 9313–9332.
- [22] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, “OmniParser for pure vision based GUI agent,” arXiv:2408.00203, 2024.
- [23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [24] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [25] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023.
- [26] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
- [27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.
- [28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. 33rd USENIX Secur. Symp. (USENIX Security)*, 2024, pp. 1831–1847.
- [29] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings Assoc. Comput. Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.
- [30] E. DeBenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.
- [31] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu *et al.*, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.
- [32] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai *et al.*, “Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution,” arXiv:2409.12191, 2024.
- [33] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.
- [34] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A survey on large language model based autonomous agents,” *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024.
- [35] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong *et al.*, “The rise and potential of large language model based agents: A survey,” *Sci. China Inf. Sci.*, vol. 68, p. 121101, 2025.